



UCCD2203 DATABASE SYSTEMS

Lecture 6

By

Mr. Cheang Kah Wai, Alvin

Department of Information Systems, FICT

Four high-level database modules

coordinates transactions on behalf of application programs.

Transaction Manager



Scheduler

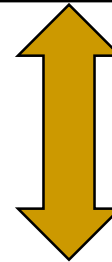
A module responsible for implementing a particular strategy for concurrency control.

Responsible for the efficient transfer of data between disk storage and main memory

Buffer Manager



Recovery Manager



- If a failure occurs during the transaction, then the database could be inconsistent.
- Recovery Manager ensures that the database is restored to the state it was in before the start of the transaction.

Transaction Manager

- Transaction Manager
coordinates transactions on
behalf of application
programs.

Buffer Manager

- Responsible for the efficient transfer of data between disk storage and main memory

The Scheduler/Lock Manager

- Establishes order of concurrent transaction execution.
- Interleaves execution of database operations to ensure serializability and isolation of transactions.
- Bases actions on concurrency control algorithms
 - Locking ,
 - Time stamping.
- Ensures efficient use of computer's CPU.
- Isolation - giving end users the impression that they are the DBMS's only users.

Recovery Manager

- If a failure occurs during the transaction, then the database could be inconsistent.
- It is the task of Recovery Manager to ensure that the database is restored to the state it was in before the start of the transaction.

Database Backup and Recovery Concept

- Back up your database
- Restoring a database
- Database recovery

Levels of backup

- A **full backup** of the database.
- **Incremental backup:**
 - Cumulative - is the backup of all files that have changed since the last full backup.
 - Differential - is the backup of the files that have changed since the last backup.

Terminology

- Backup strategy may include:
 - Entire database (**whole**)
 - Portion of the database (**partial**)
- Backup type/level may indicate inclusion of:
 - All data blocks within your chosen files (**full**)
 - Only information that has changed since a previous backup(**incremental**)
 - Cumulative (changes since last level 0)
 - Differential (changes since last incremental-level 0 or 1 parents)
- Backup mode may be:
 - **Offline** (consistent, cold)
 - **Online** (inconsistent, hot)

Incremental Backups of Data Files

- Incremental backups, copy only those blocks in a data file that change between backups.
- level 0 incremental backup copy all blocks in the data file, used as the starting point
- Incremental backups at level 1 copy only images of blocks that have changed since a previous incremental backup.
- Cumulative; Differential

Consistent and Inconsistent Backup

- consistent backup, your database must have been shut down cleanly and remain closed for the duration of the backup.
- Open database backups are inconsistent because the online redo log files contain changes not yet applied to the data files. Database must be in ARCHIVELOG mode

Database Recovery

- Via Reprocessing
 - Is this possible? Feasible?
 - Those transactions were run concurrently
- Via Rollback/Rollforward

Recovery Terminology

- **Log** - records of the data changes in chronological order
- **Change log/ Before and After-images** copy of every record before / after it was changed
- **Transaction log** – keeps a record of all changes
- **Checkpoint** - a point of synchronization between the database and the transaction log

Database Recovery

Process of restoring database to a correct state in the event of a failure.

- Need for Recovery Control
 - ❑ Two types of data storage: volatile (main memory) and nonvolatile.
 - ❑ Volatile storage does not survive system crashes.
 - ❑ Stable storage represents information that has been replicated in several nonvolatile storage media with independent failure modes.

Categories of Failure

1. Statement failure: A single database operation (select, insert, update, or delete) fails.
2. User process failure: A single database session fails.
3. Network failure: Connectivity to the database is lost.
4. User error: A user successfully completes an operation, but the operation (dropping a table or entering bad data) is incorrect.
5. Instance failure: The database instance shuts down unexpectedly.
6. Media failure: A loss of any file that is needed for database operation (that is, the files have been deleted or the disk has failed).

1. Statement Failure

Typical problems:

- ❑ Attempts to enter invalid data into a table
- ❑ Attempts to perform operations with insufficient privileges
- ❑ Attempts to allocate space that fail
- ❑ Logic errors in applications

2. User Process Failure

Typical problems:

- ❑ A user performs an abnormal disconnect.
- ❑ A user's session is abnormally terminated.
- ❑ A user experiences a program error that terminates the session.

3. Network Failure

Typical problems:

- ❑ Listener fails.
- ❑ Network Interface Card (NIC) fails.
- ❑ Network connection fails.

4. User Error

- Typical problems:
 - User inadvertently deletes or modifies data.
 - User drops a table.

5. Instance Failure

Typical problems:

- ❑ Power outage
- ❑ Hardware failure
- ❑ Failure of one of the critical background processes
- ❑ Emergency shutdown procedures

6. Media Failure

Typical problems:

- ❑ Failure of disk drive
- ❑ Failure of disk controller
- ❑ Deletion or corruption of a file needed for database operation

Types of failures

- Disaster
 - Natural physical disasters.
 - Carelessness or unintentional destruction of data or HW/SW facilities.
 - Sabotage.

Recovery Facilities

- DBMS should provide following facilities to assist with recovery:
 - *Backup mechanism*, which makes periodic backup copies of database.
 - *Logging/journalizing facilities*, which keep track of current state of transactions and database changes.
 - *Checkpoint facility*, which enables updates to the database in progress to be made permanent.
 - *Recovery manager*, which allows DBMS to restore database to consistent state following a failure.

Backup mechanism

- The DBMS should provide a mechanism to allow backup copies of the database and the *log file* to be made at regular intervals without necessarily having to stop the system first.
- The backup copy of the database can be used in the event that the database has been damaged or destroyed.
- A backup can be a complete copy of the entire database or an incremental backup (modifications made since the last complete).
- Backup can be stored on offline storage, such as optical disk.

Log File (redo log file)

- Contains information about all updates to database:
 - Transaction records.
 - Checkpoint records.
- Often used for other purposes (for example, auditing).
- Recommended to store on a disk separate from the main database files

Log File

Transaction records contain:

- ❑ Transaction identifier;
- ❑ Type of log record, (transaction start, insert, update, delete, abort, commit);
- ❑ Identifier of data item affected by database action (insert, delete, and update operations);
- ❑ Before-image of data item;
- ❑ After-image of data item;
- ❑ Log management information.

Sample log file – in Next slide

Sample Log File

Tid	Time	Operation	Object	Before image	After image	pPtr	nPtr
T1	10:12	START				0	2
T1	10:13	UPDATE	STAFF SL21	(old value)	(new value)	1	8
T2	10:14	START				0	4
T2	10:16	INSERT	STAFF SG37		(new value)	3	5
T2	10:17	DELETE	STAFF SA9	(old value)		4	6
T2	10:17	UPDATE	PROPERTY PG16	(old value)	(new value)	5	9
T3	10:18	START				0	11
T1	10:18	COMMIT				2	0
	10:19	CHECKPOINT	T2, T3				
T2	10:19	COMMIT				6	0
T3	10:20	INSERT	PROPERTY PG4		(new value)	7	12
T3	10:21	COMMIT				11	0

Figure 22.25, Pg: 657

Difficulties in logging facilities

- When a failure occurs, we may not know how far back in the log to search, and we may end up redoing transactions that have been safely written to the database.

Checkpointing

Checkpoint

Point of synchronization between database and log file. All buffers are force-written to secondary storage.

Checkpoints

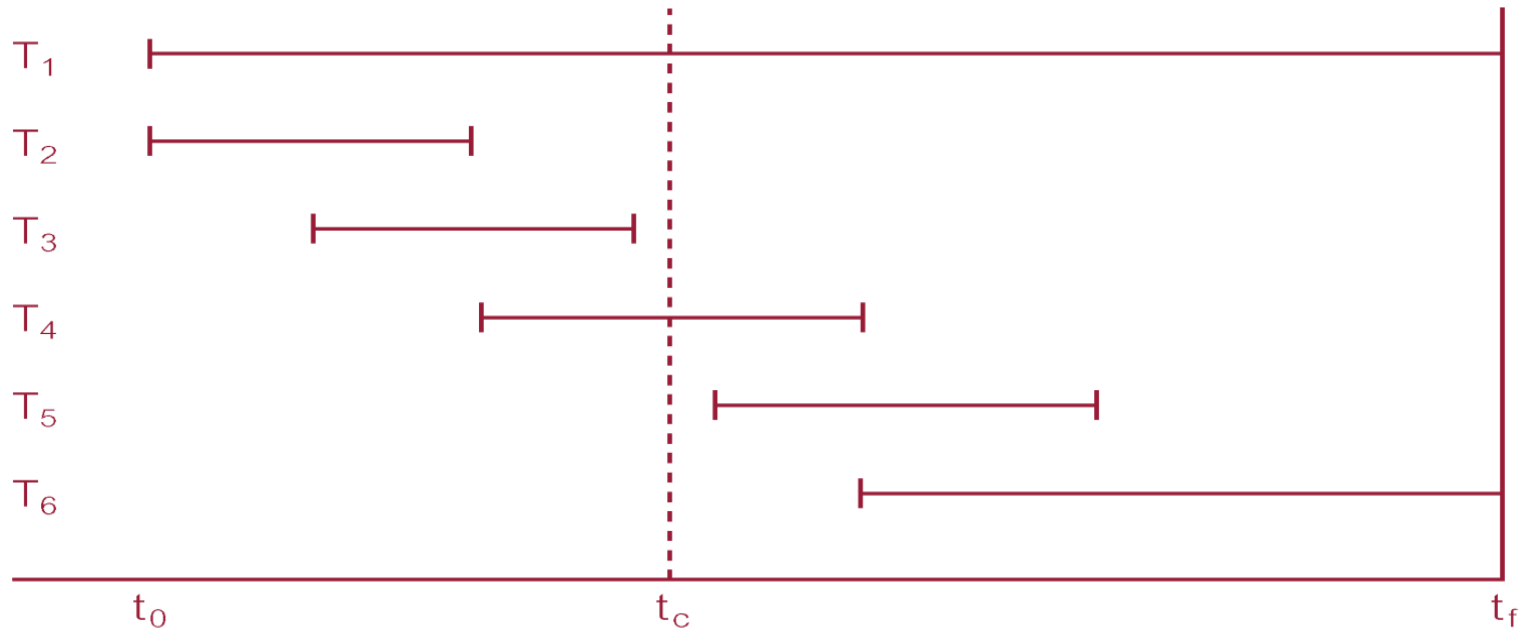
Checkpoints are scheduled at predetermined interval and involve the following operations:

- ❑ Writing all log records in main memory to secondary storage;
- ❑ Writing the modified blocks in the database buffers to secondary storage;
- ❑ Writing a checkpoint record to the log file. This record contains the identifiers of all transactions that are active at the time of checkpoint.

Checkpoints

- If transactions are performed serially, when a failure occurs we check the log file to find the last transaction that started before the last checkpoint.
- If the transaction is active at the time of failure, the transaction must be *undone*.
- When failure occurs, *redo* all transactions that committed since the checkpoint and undo all transactions active at time of crash.

Example



- DBMS starts at time t_0 , but fails at time t_f . Assume data for transactions T_2 and T_3 have been written to secondary storage.

Checkpointing

- In previous example, with checkpoint at time t_c , changes made by T_2 and T_3 have been written to secondary storage.
- Thus:
 - redo T_4 and T_5 ,
 - undo transactions T_1 and T_6 .

Recovery techniques

- Recovery manager is responsible for atomicity and durability.
- The type of restart used depends on the nature of the failure and the extent of damage.
- If failure occurs between commit and database buffers being flushed to secondary storage then, to ensure durability, recovery manager has to *redo* (*rollforward*) transaction's updates.

Recovery techniques

If transaction had not committed at failure time, recovery manager has to *undo (rollback)* any effects of that transaction for atomicity.

- **Partial undo** - only one transaction has to be undone.
- **Global undo** - all transactions have to be undone.

Recovery Techniques

- If database has been damaged:
 - ❑ Need to restore last backup copy of database and reapply updates of committed transactions using log file.
- If database is only inconsistent:
 - ❑ Need to undo changes that caused inconsistency. May also need to redo some transactions to ensure updates reach secondary storage.
 - ❑ Do not need backup, but can restore database using *before- and after-images* in the log file.

Recovery Techniques: Forward Recovery

- Recreate the database after the crash.
- Begin by reading the last backup copy that was made
- Reading the log with all the changes since the last backup copy was made.
- The recovery manager updates the backup copy of the table with the logs.
- When the process is completed, the lost table is rebuilt.

REDO

Recovery Techniques: Backward Recovery

- An error is discovered that involves a piece of recently updated data.
- Other programs may have read the incorrect data and made use of it.
- Therefore, the discovered error, and all other changes that were made to the database since the error was found must be rollback.
 - Use the before and after image in log file

UNDO

Main Recovery Approaches

Three main recovery approaches:

- ❑ Deferred Update,
- ❑ Immediate Update, and
- ❑ Shadow Paging.

Recovery techniques using deferred update

- Updates are not written to the database until after a transaction has reached its commit point.
- If transaction fails before commit point, it will not have modified database and so no undoing of changes required.
- May be necessary to redo updates of committed transactions as their effect may not have reached database.

Recovery techniques using deferred update

We use the log file to protect against system failures in the following ways:

- When a transaction starts, write a *transaction start* record to the log.
- When any write operation is performed, write a log record containing the after-image of the update.
- When a transaction is about to commit:
 - write a Transaction Commit log record,
 - write all the log records for the transaction to disk, then
 - Use the log records to perform the actual updates to the database.
- If a transaction aborts, write a *transaction abort* log record and do not perform the writes,
 - A transaction without a Transaction Abort record is also aborted.

Recovery techniques using immediate update

- Updates are applied to database as they occur.
- Need to redo updates of committed transactions following a failure.
- May need to undo effects of transactions that had not committed at time of failure.

Recovery techniques using immediate update

We use the log file to protect against system failures in the following ways:

- When a transaction starts, write a *transaction start* record to the log.
- When any write operation is performed, write a log record containing the necessary data to the log file.
- When the log record is written, write the update to the database buffers.
- The updates to the database itself are written when the buffers are next flushed to secondary storage.
- When the transaction commits, write a *transaction commit* record to the log.

Immediate Update-*Write-ahead log protocol*

It is essential that log records are written before write to database. *Write-ahead log protocol*.

- Log record writes must preceded database writes.
- If updates were made to the database first, and failure occurred before the log record was written, then RMAN (Recovery Manager) would have no way of undoing or redoing the operation.

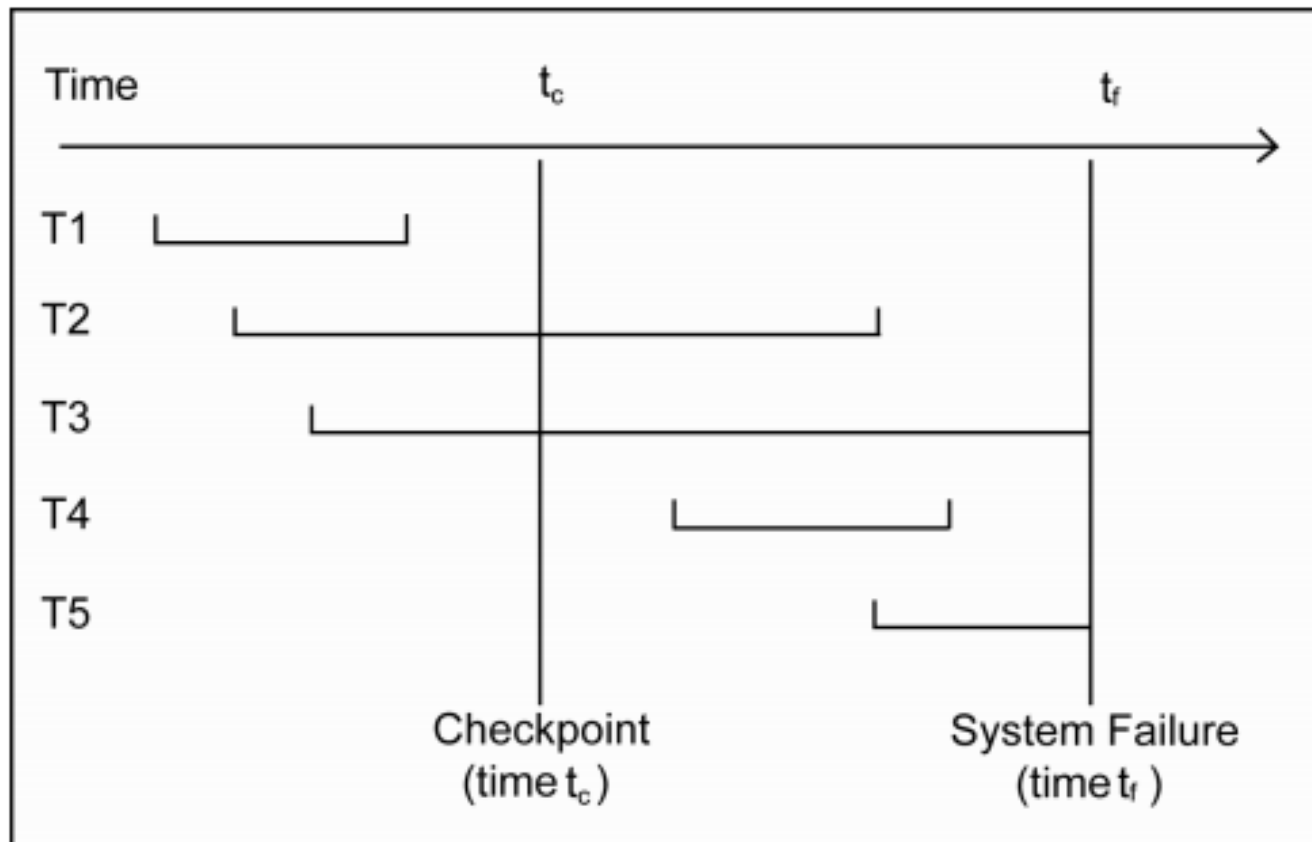
Recovery techniques using immediate update

- If no "*transaction commit*" record in log, then that transaction was active at failure and must be undone.
- Undo operations are performed *in reverse order in which they were written to log*.

Shadow Paging

- Maintain two page tables during life of a transaction: *current* page and *shadow* page table.
- When transaction starts, two pages are the same.
- Shadow page table is never changed thereafter and is used to restore database in event of failure.
- During transaction, current page table records all updates to database.
- When transaction completes, current page table becomes shadow page table.

Activity: 1



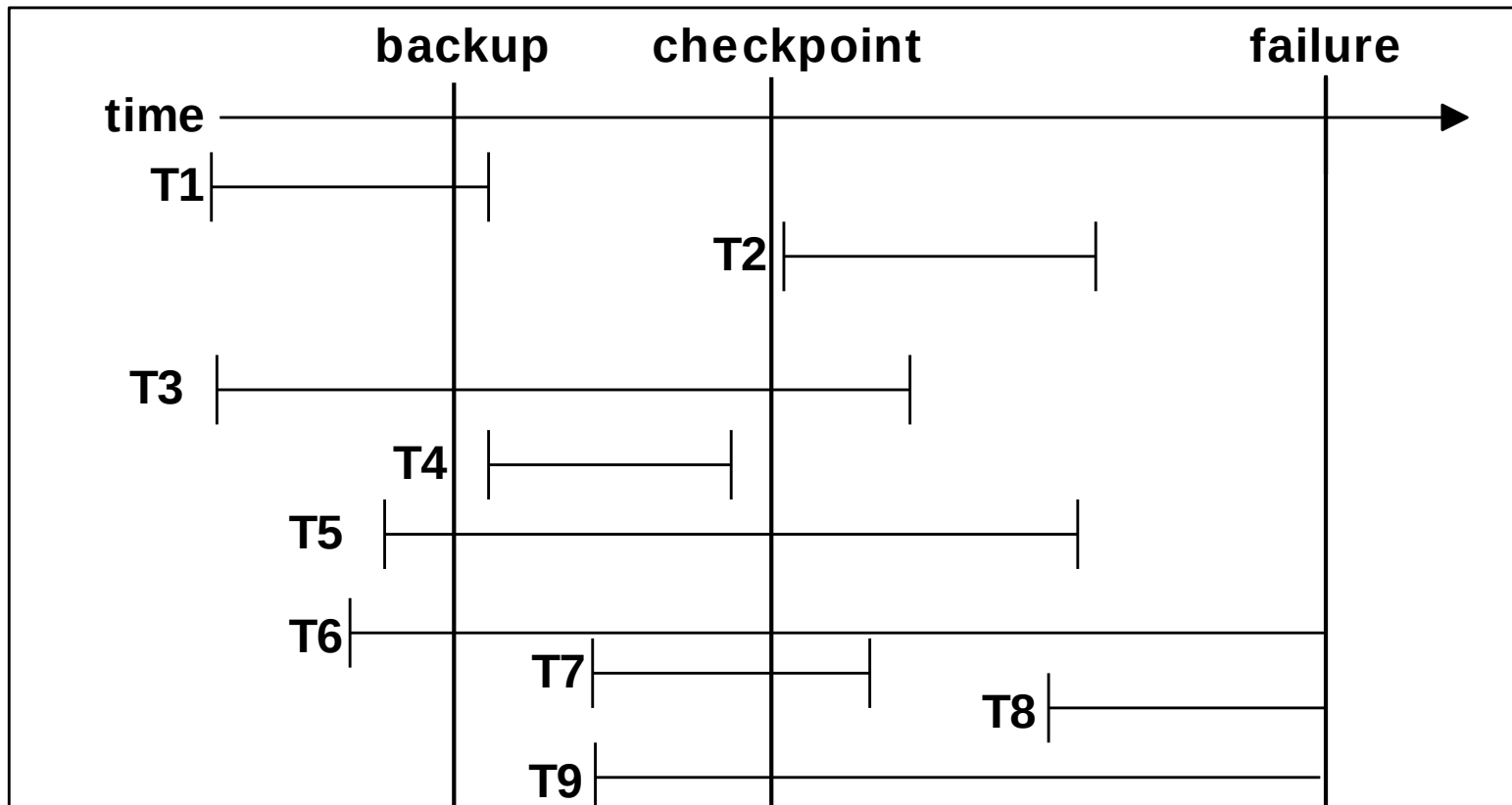
Immediate update approach

Transaction ID	Description	Operation
T1	Finished before CP	None
T2	Started before CP finished before failure	Redo
T3	Started before CP not yet finished	Undo
T4	Started after CP finished before failure	Redo
T5	Started after CP not yet finished	Undo

Deferred update approach

Transaction ID	Description	Operation
T1	Finished before CP	None
T2	Started before CP finished before failure	Redo
T3	Started before CP not yet finished	None
T4	Started after CP finished before failure	Redo
T5	Started after CP not yet finished	None

Activity 2:



1. Immediate update approach

- T1: None,
- T2: Redo forward from checkpoint,
- T3: Redo forward from checkpoint,
- T4: None,
- T5: Redo forward from checkpoint,
- T6: Undo backward from most recent log record,
- T7: Redo forward from checkpoint,
- T8: Undo backward from most recent log record,
- T9: Undo backward from most recent log record,

2. Deferred update approach

- T1: None
- T2: Redo forward from the first log record
- T3: Redo forward from the first log record
- T4: None
- T5: Redo forward from the first log record
- T6: None
- T7: Redo forward from the first log record
- T8: None
- T9: None

3. restart work for each transaction if a device failure occurs affecting the database table

- T1: Redo forward from backup
- T2: Redo forward from backup
- T3: Redo forward from backup
- T4: Redo forward from backup
- T5: Redo forward from backup
- T6: Undo backward from most recent log record, restarts incomplete transactions
- T7: Redo forward from backup
- T8: Undo backward from most recent log record, restarts incomplete transactions
- T9: Undo backward from most recent log record, restarts incomplete transactions

The End Of Lecture 6

Thank You